

Cloud Computing (cloud)

Module 8: Container Orchestration Addons (Kubernetes)

Prof. Dr. Sebastian Graf (sebastian.graf@fhnw.ch)

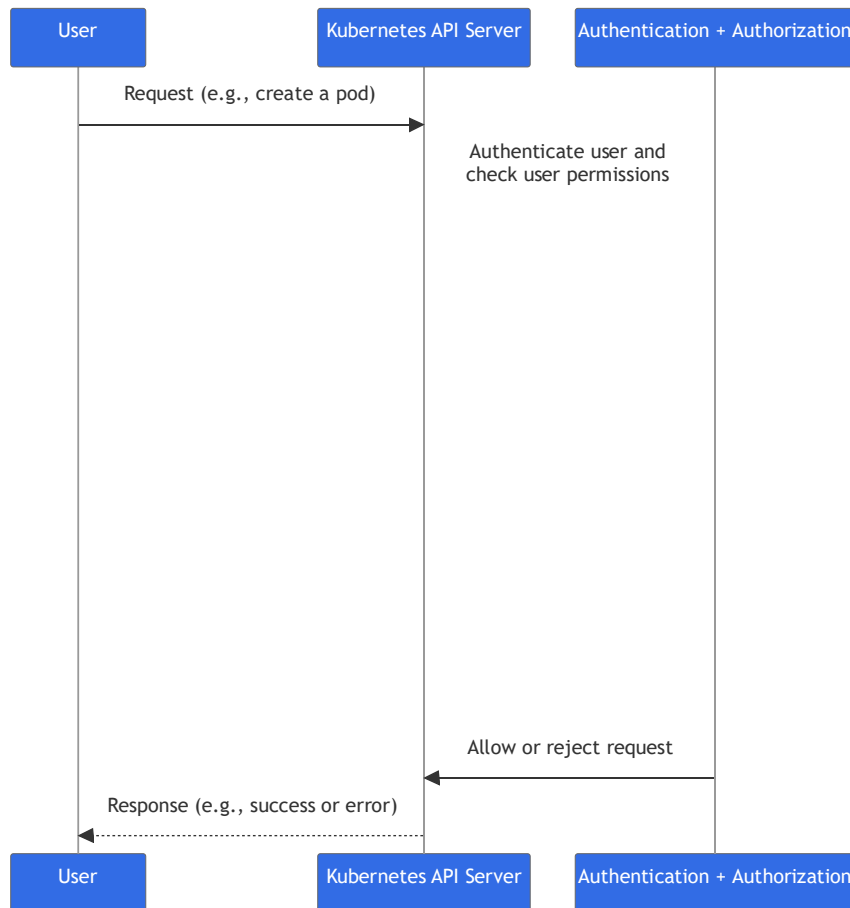
Norwin Schnyder (norwin.schnyder@fhnw.ch)



Input of this Module

- <https://learning.oreilly.com/playlists/43827098-7a87-435e-823e-736586b5694c> (08-containerorchaddons)
- Kubernetes Documentation: <https://kubernetes.io/docs/concepts/>
- Several Blogposts

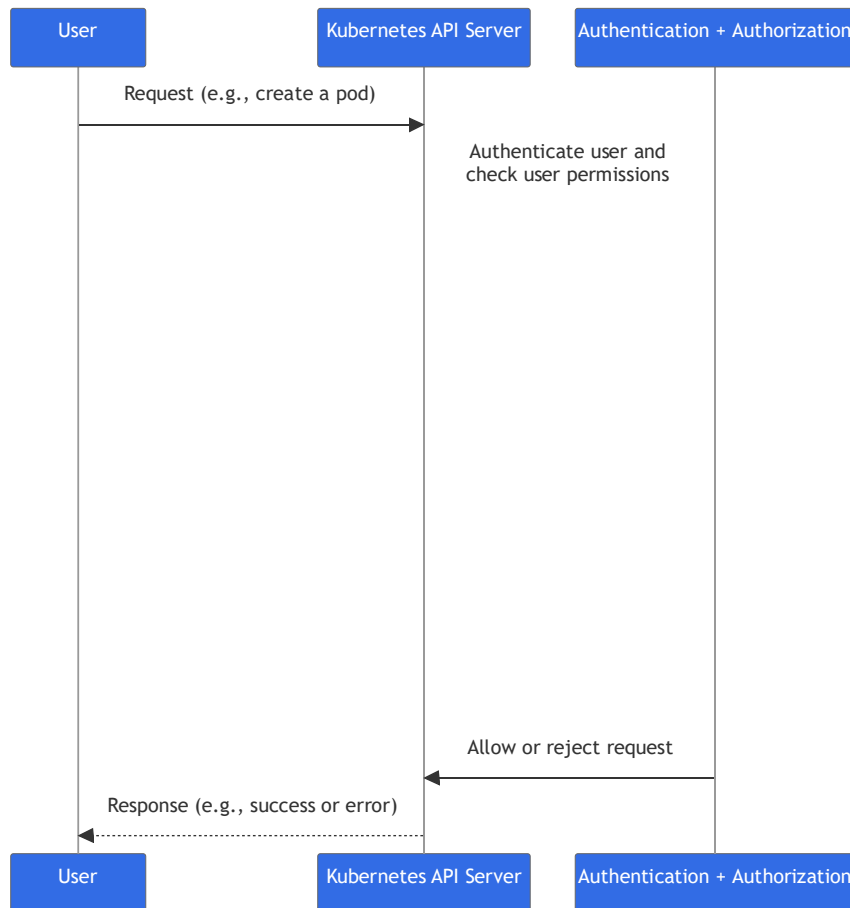
Authentication, Authorization



Authentication

- Authenticate a user based on credentials provided as part of API requests.
- Kubernetes supports two kinds of users:
 - **Normal Users** (external users) managed outside of the Kubernetes cluster via independent services (e.g., OIDC)
 - **Service Accounts** (internal users) allow in-cluster processes to communicate with the API server using a token

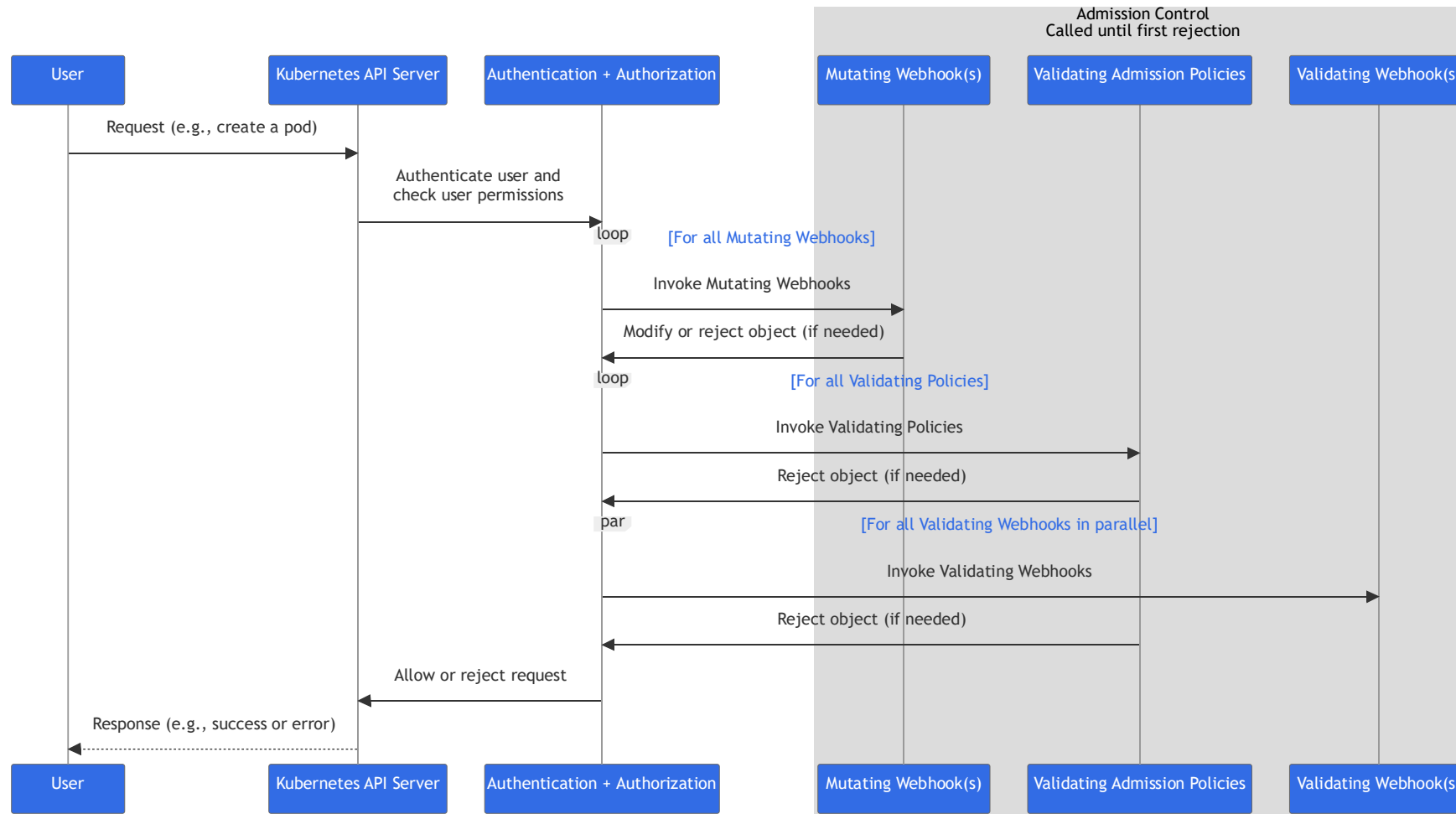
Authentication, Authorization



Authorization

- Authorizes the API requests submitted by the authenticated user.
- In general Kubernetes is using **Role-Based Access Control (RBAC)** to regulate the access to resources based on the Roles of individual users.
 - Roles must be defined according to the least privileges principle.
 - There are two kinds of roles: `Role` and `ClusterRole`
 - Roles are bind to users (e.g., service accounts) using role bindings (`RoleBinding` and `ClusterRoleBinding`)

Authentication, Authorization, and Admission Control



Admission controllers are used to specify granular access control policies, which include allowing privileged containers, checking on resource quota, etc.

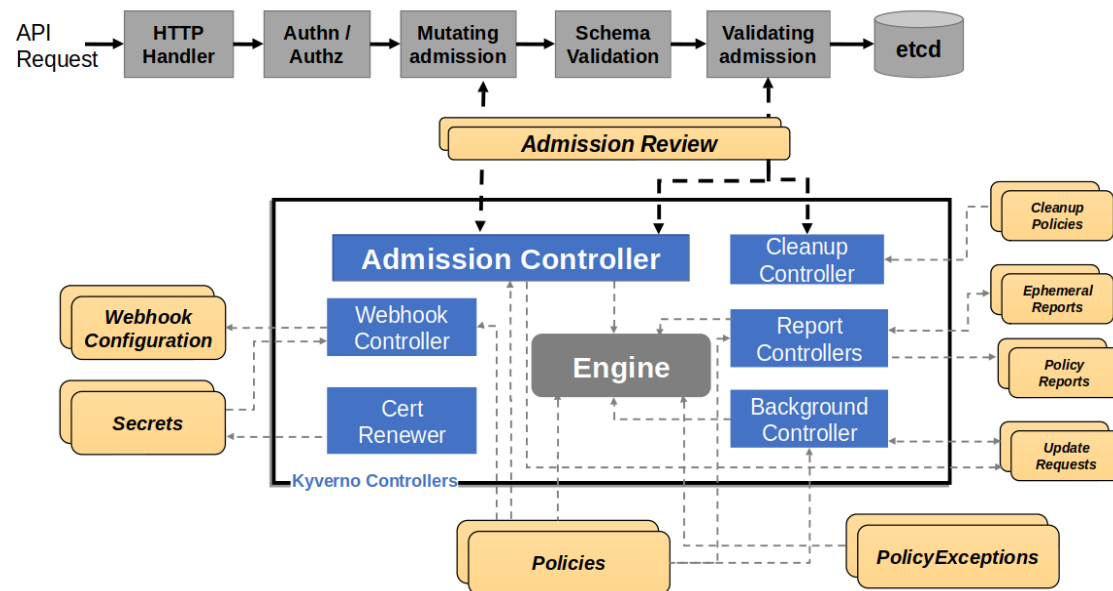
Example:





Admission Control: Kyverno

Kyverno runs as a dynamic admission controller in a Kubernetes cluster. Kyverno receives validating and mutating admission webhook HTTP callbacks from the Kubernetes API server and applies matching policies to return results that enforce admission policies or reject requests.



```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: allowed-image-repos
  annotations:
    policies.kyverno.io/title: Allowed Image Repositories
    policies.kyverno.io/category: Other
    policies.kyverno.io/severity: medium
    kyverno.io/kyverno-version: 1.9.0
    kyverno.io/kubernetes-version: "1.24"
    policies.kyverno.io/subject: Pod
    policies.kyverno.io/description: >-
      In addition to restricting the image registry from which images are pulled, in some cases
      and environments it may be required to also restrict which image repositories are used,
      for example in some restricted Namespaces. This policy ensures that the only allowed
      image repositories present in a given Pod, across any container type, come from the
      designated list.
spec:
  validationFailureAction: Audit
  background: false
  rules:
    - name: good-repos
      match:
        any:
          - resources:
              kinds:
                - Pod
      validate:
        message: >-
          All images in this Pod must come from an authorized repository.
        deny:
          conditions:
            all:
              - key: "{{ images.containers, initContainers, ephemeralContainers[] }.*.name[] }}"
                operator: AnyNotIn
                value:
                  - myknownimage
                  - kyverno
```

Extending Kubernetes: Plugin Model

– CRI

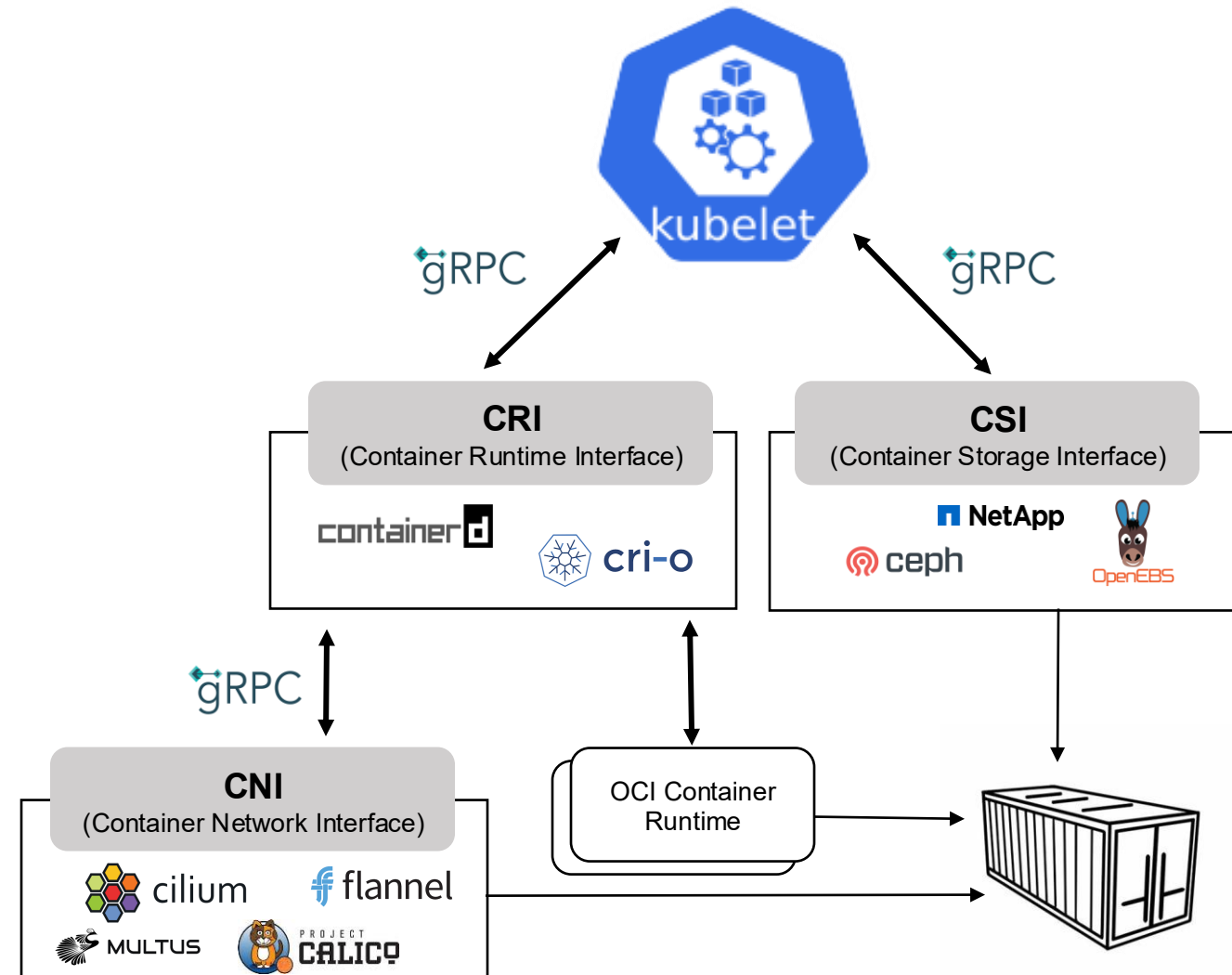
- different runtimes supported such as docker, cri-o, containerd, etc.

– CNI

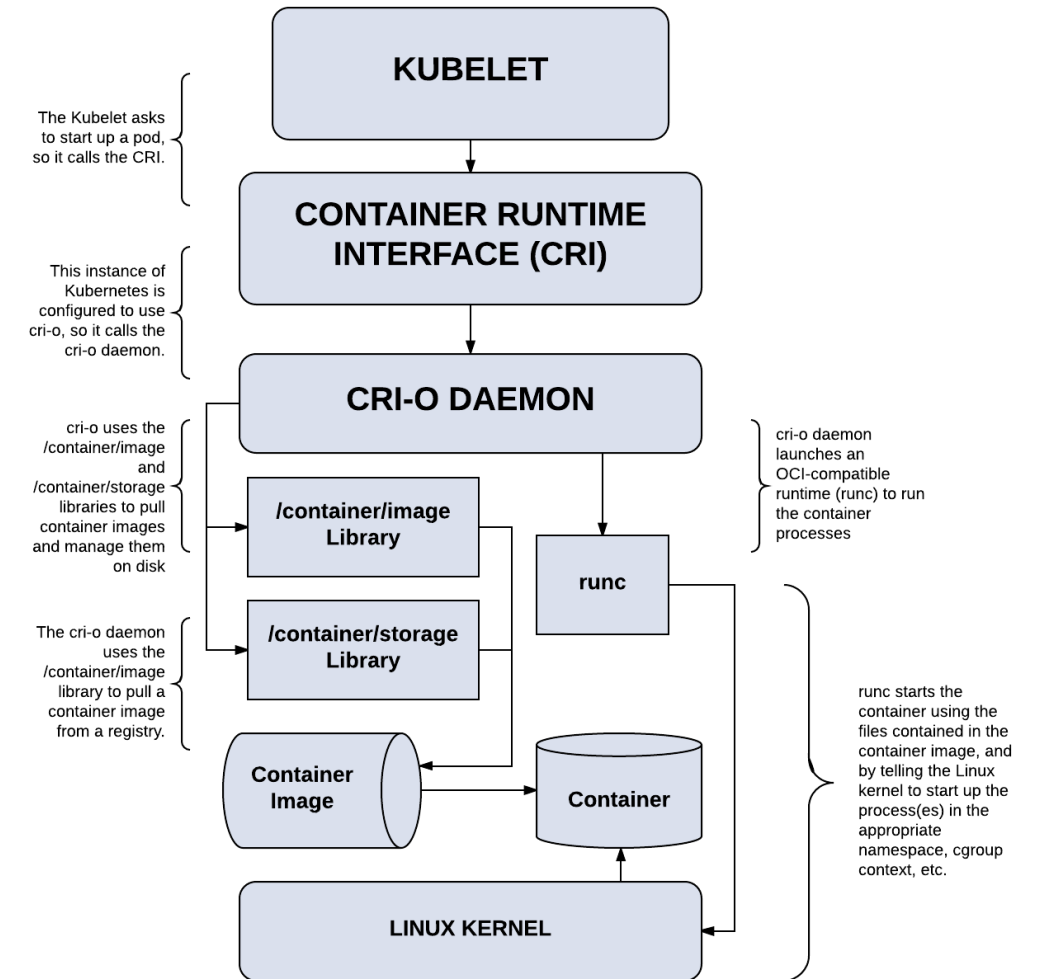
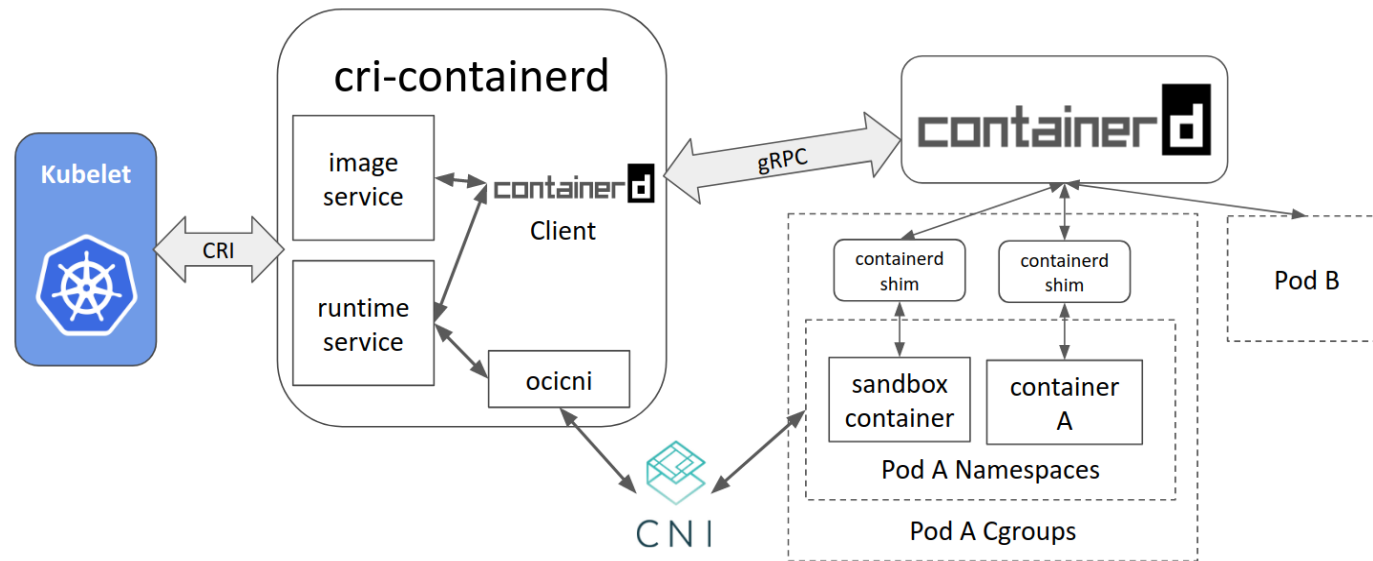
- Software-defined Networking must be defined to ensure access to pods

– CSI

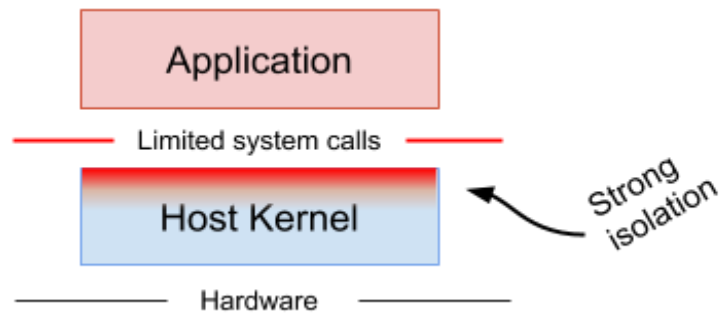
- Storage interface to provision different storage kinds for persistence



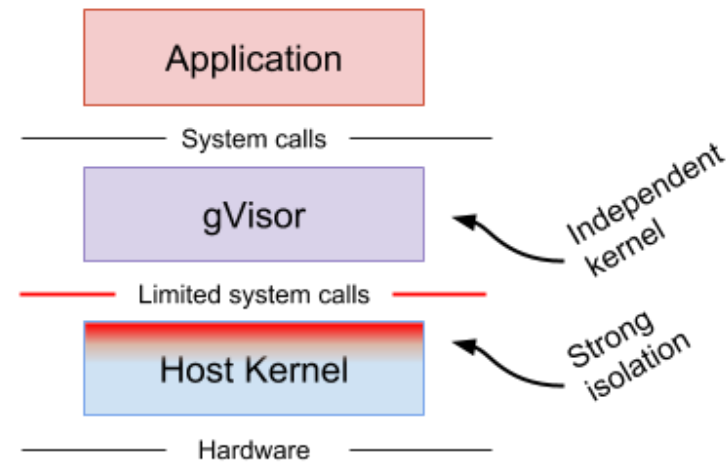
Container Runtime Interface: Examples



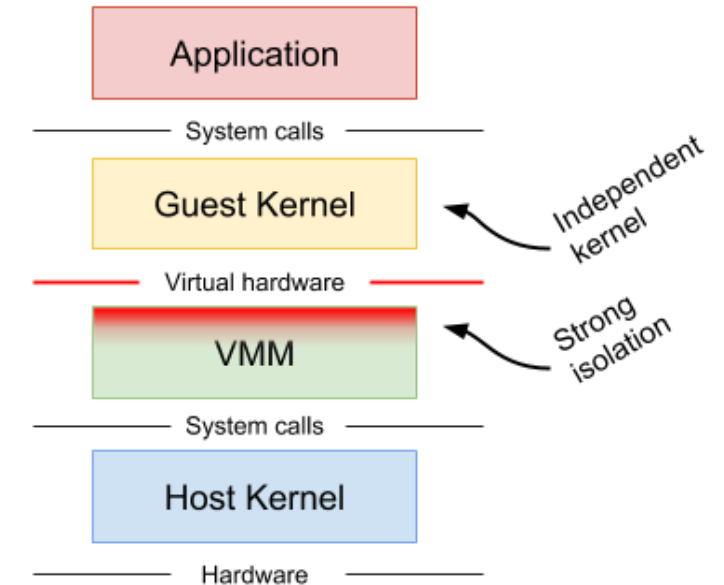
Isolating Container Runtimes



Rule-based execution
seccomp, SELinux and AppArmor



Intercepted syscalls
gVisor



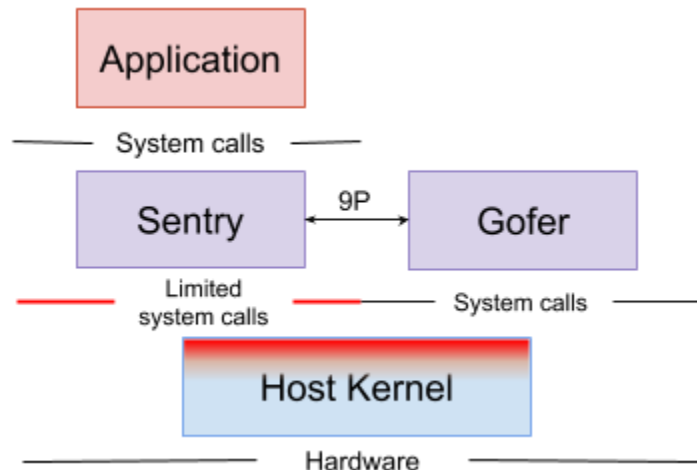
Machine-level virtualization
Kata Containers
(KVM, Firecracker)



Isolating Container Runtimes: gVisor

gVisor provides a strong layer of isolation between running applications and the host operating system. It is an application kernel that implements a Linux-like interface.

gVisor includes an **Open Container Initiative (OCI)** runtime called **runsc** that makes it easy to work with existing container tooling. The **runsc** runtime integrates with Docker and Kubernetes (e.g., containerd, cri-o), making it simple to run sandboxed containers.



Each sandbox has its own isolated instance of:

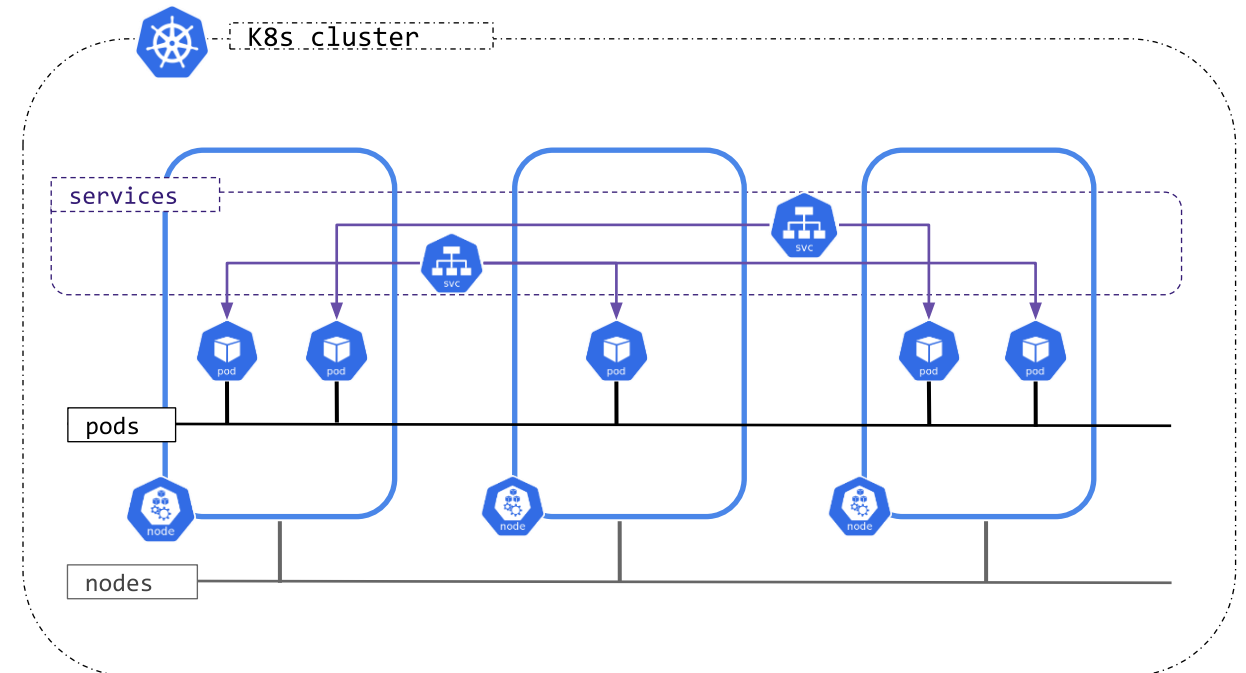
- The **Sentry**, which is a kernel that runs the containers and intercepts and responds to system calls made by the application.

Each container running in the sandbox has its own isolated instance of:

- A **Gofer** which provides file system access to the containers.

Container Network Interface

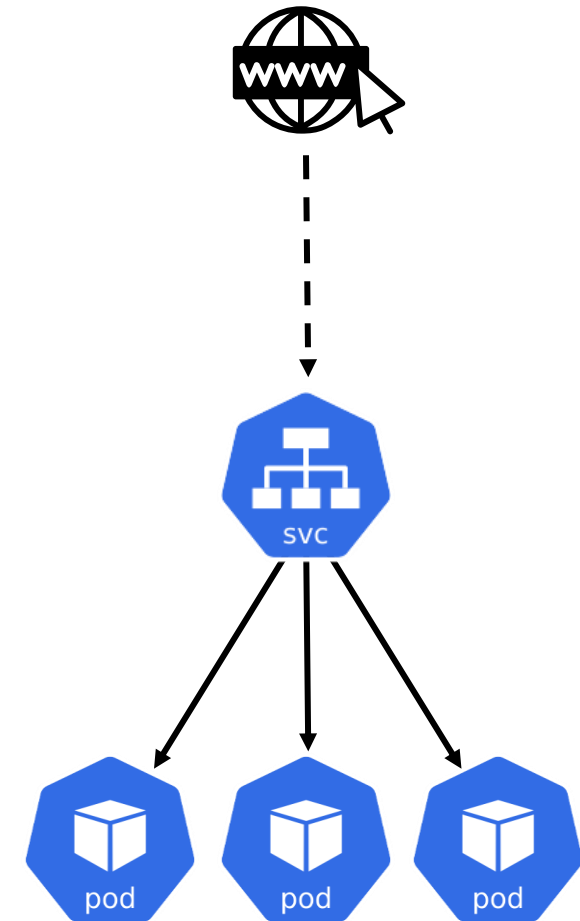
- A pod has its own private network namespace which is shared by all of the containers within the pod.
- Each pod in a cluster gets its own unique cluster-wide IP address.
- IP-ranges of pods/services must be independent of IP-ranges of nodes
- SDN (Software defined networking) encapsulates node-spanning access from one pod to another on different nodes
- Different implementations exists, some of them can be combined



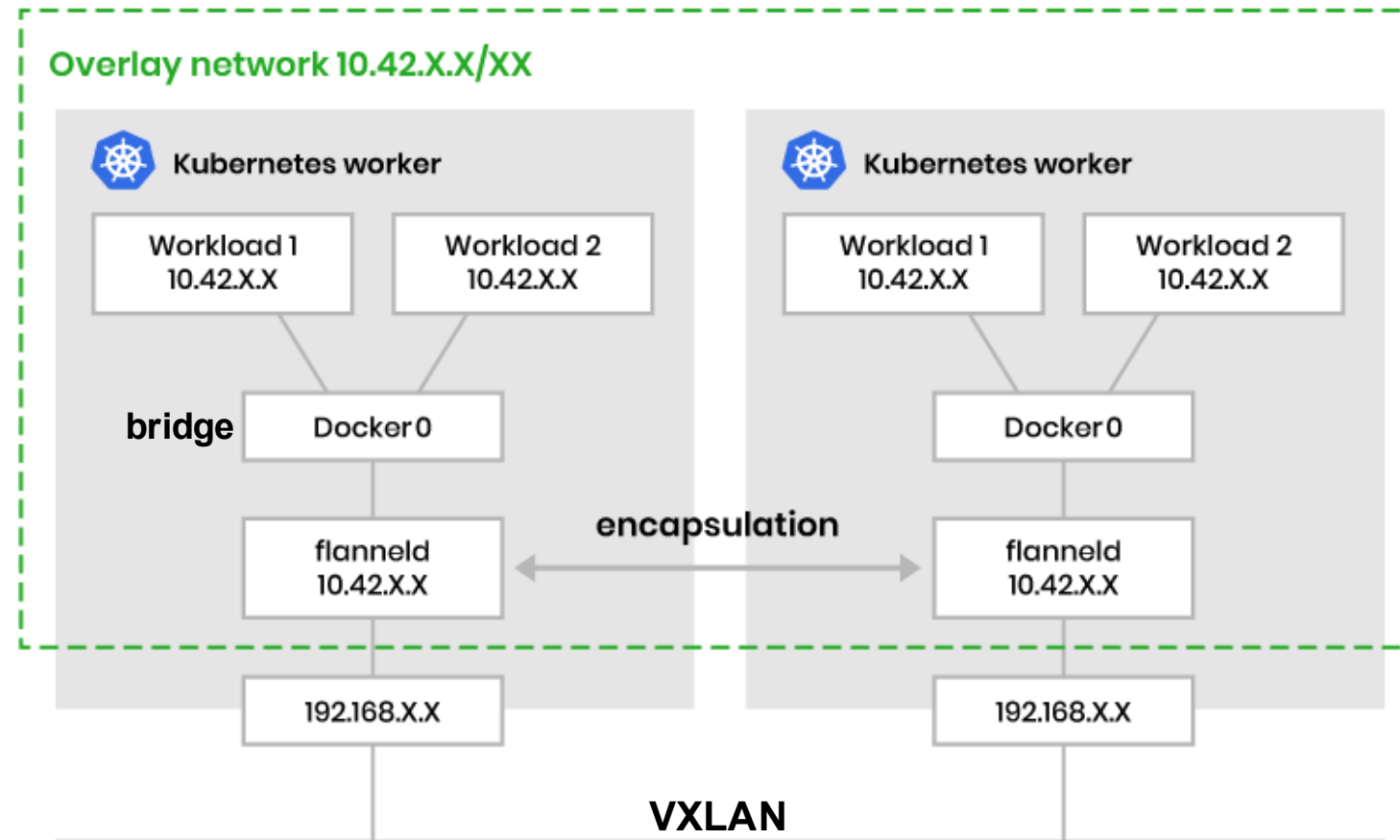
Note: Because the cluster is not routable from the broader network, multiple clusters can even use the same IP address space.

Kubernetes Network Model

- The Service API lets you provide a stable (long lived) IP address or hostname for a service implemented by one or more backend pods, where the individual pods making up the service can change over time.
 - Kubernetes automatically manages `EndpointSlice` objects to provide information about the pods currently backing a Service.
- Kubernetes Service types allow you to specify what kind of Service you want:
 - `ClusterIP` - Exposes the Service on a cluster-internal IP.
 - `NodePort` - Exposes the Service on each Node's IP at a static port.
 - `LoadBalancer` - Exposes the Service externally using an external load balancer.
- The Gateway API for Kubernetes provides extra capabilities beyond Service.



Container Network Interface: Examples

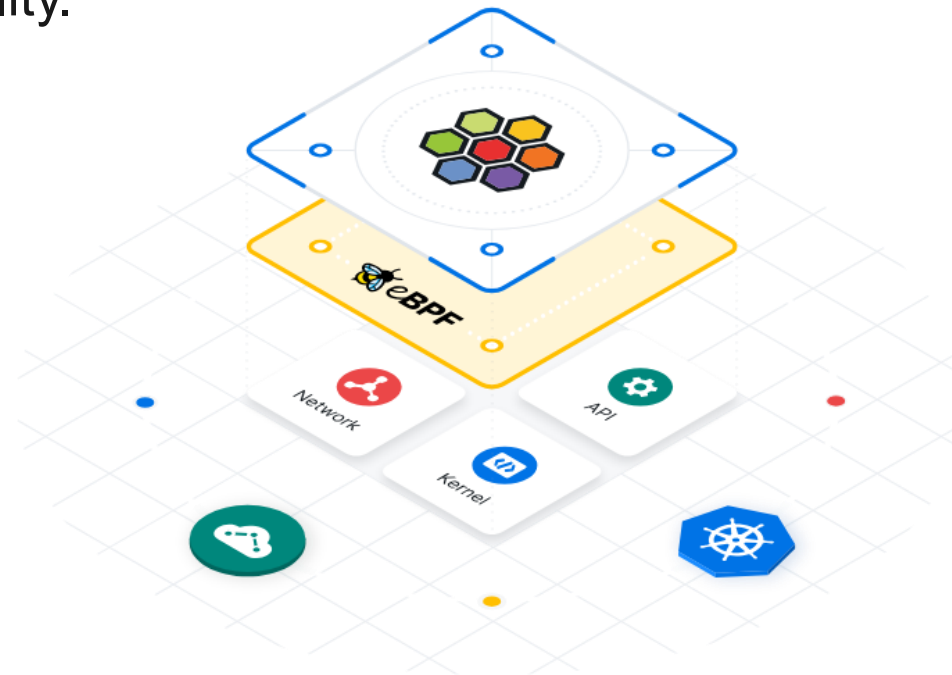




Container Network Interface: Examples

Cilium is an open-source project to provide networking, security, and observability for cloud native environments such as Kubernetes clusters and other container orchestration platforms.

At the foundation of Cilium **eBPF** is used to provide high-performance networking, multi-cluster and multi-cloud capabilities, advanced load balancing, transparent encryption, extensive network security capabilities, transparent observability.

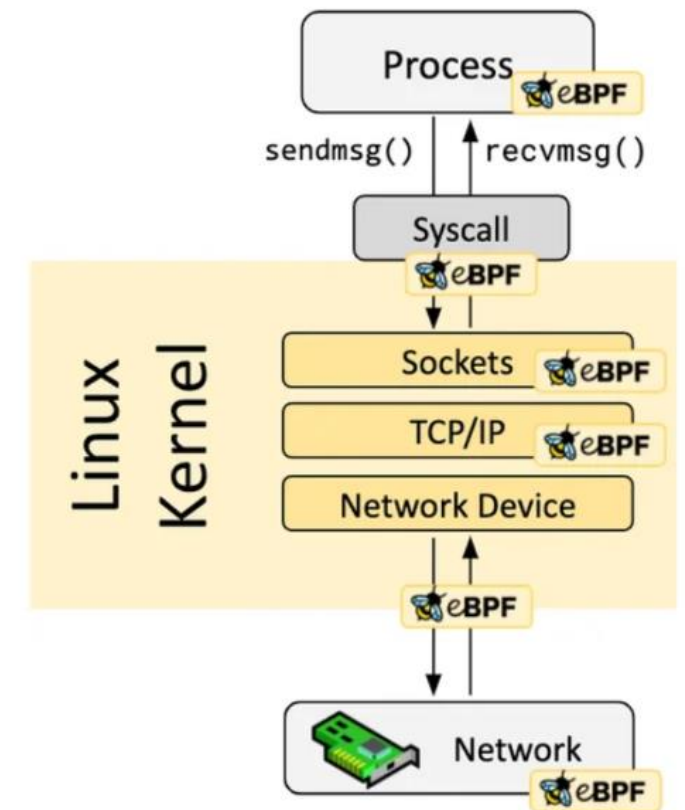


Cilium: What is eBPF?

eBPF (extended Berkeley Packet Filter) can run sandboxed programs in a privileged context such as the operating system kernel. It is used to safely and efficiently extend the capabilities of the kernel without requiring to change kernel source code or load kernel modules.

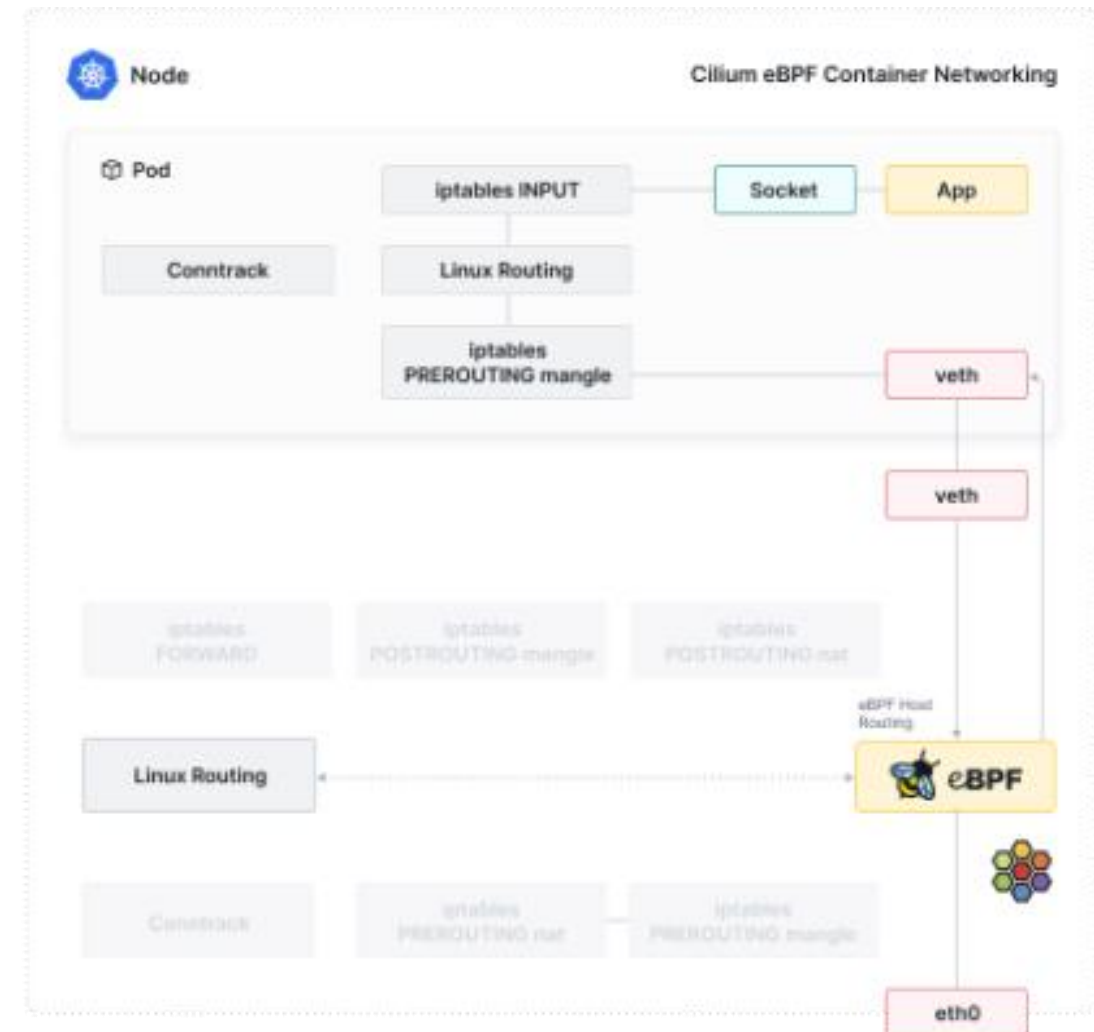
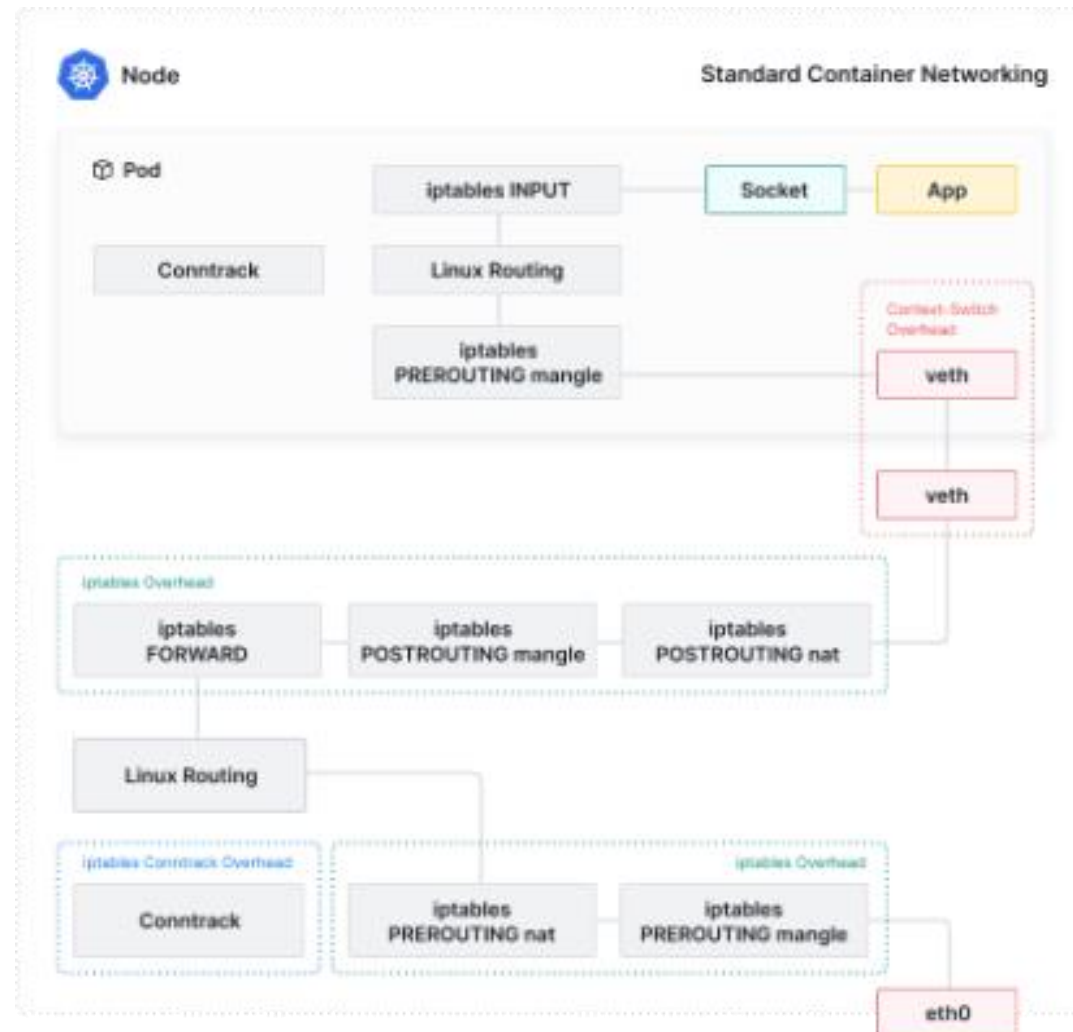
eBPF programs have the capability to **customize the networking behavior**:

- Using a return code from the eBPF program to tell the kernel what to do with a network packet (processing it as usual, dropping it, or redirecting it to a different destination)
- Allowing the eBPF program to modify network packets, socket configuration





Container Network Interface: Examples

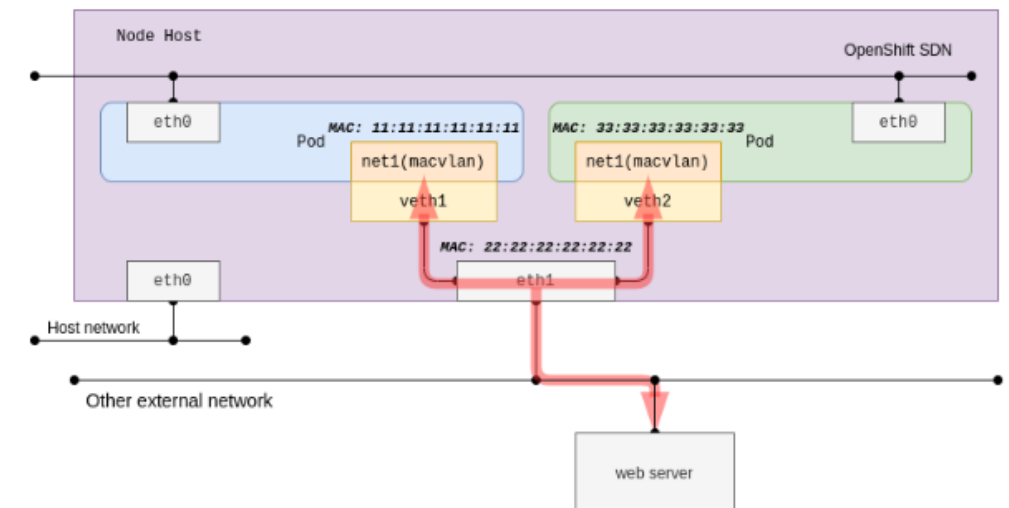
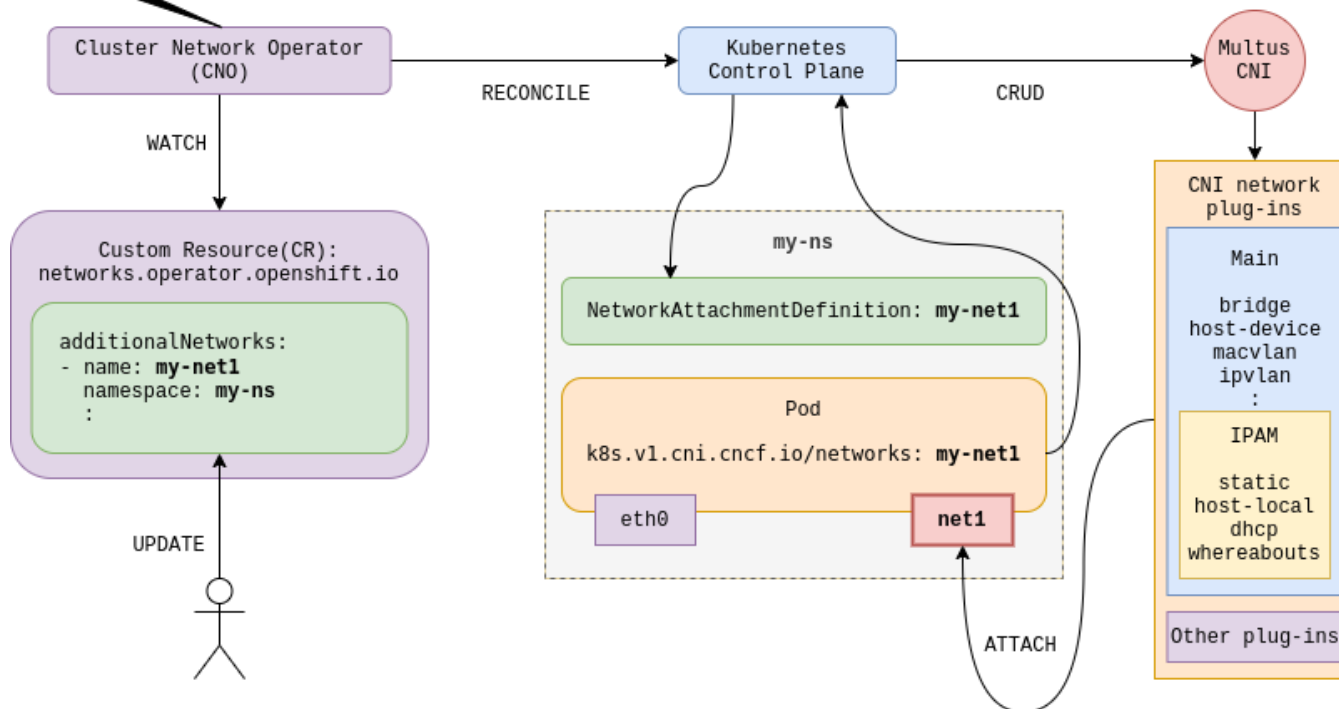




Container Network Interface: Examples

Multus is a CNI Plugin which enables attaching multiple network interfaces to pods. Typically, in Kubernetes each pod only has one network interface (apart from a loopback).

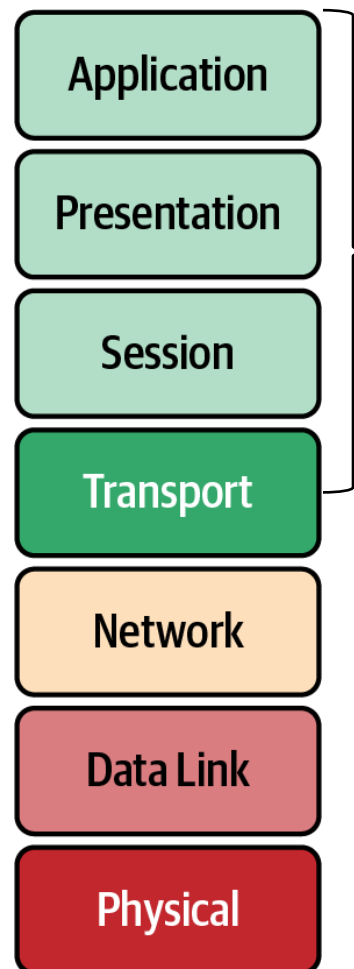
```
reconciling (k8s.cni.cncf.io/v1, Kind=NetworkAttachmentDefinition) my-ns/my-net1
does not exist, creating (k8s.cni.cncf.io/v1, Kind=NetworkAttachmentDefinition) my-ns/my-net1
successfully created (k8s.cni.cncf.io/v1, Kind=NetworkAttachmentDefinition) my-ns/my-net1
```



Kubernetes Network Model

The **CNI Plugin** is responsible that:

- Each pod in a cluster gets its own unique cluster-wide IP address.
- The *pod network* (also called a cluster network) handles communication between pods.
 - All pods can communicate with all other pods, whether they are on the same node or on different nodes.



gateway api

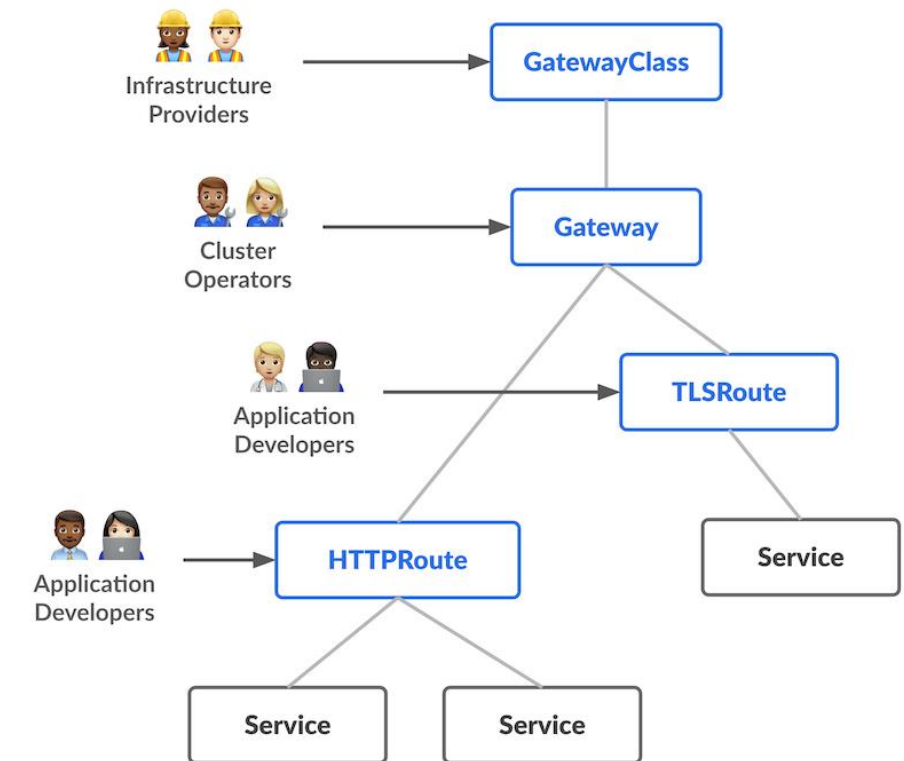
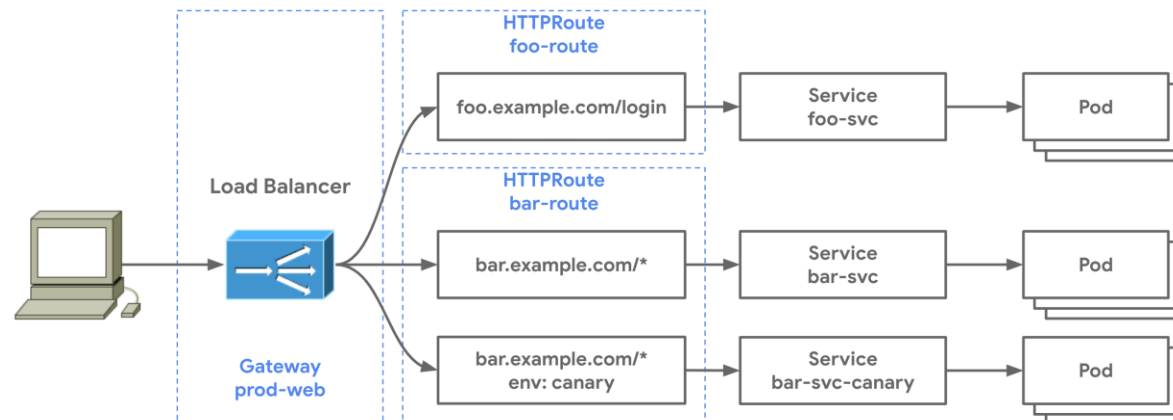
Gateway API is an official Kubernetes project focused on L4 and L7 routing in Kubernetes. This project represents the next generation of Kubernetes **Ingress** (north/south traffic) and **Service Mesh** (east/west traffic) APIs. It allows you to make Services accessible to clients that are outside the cluster.





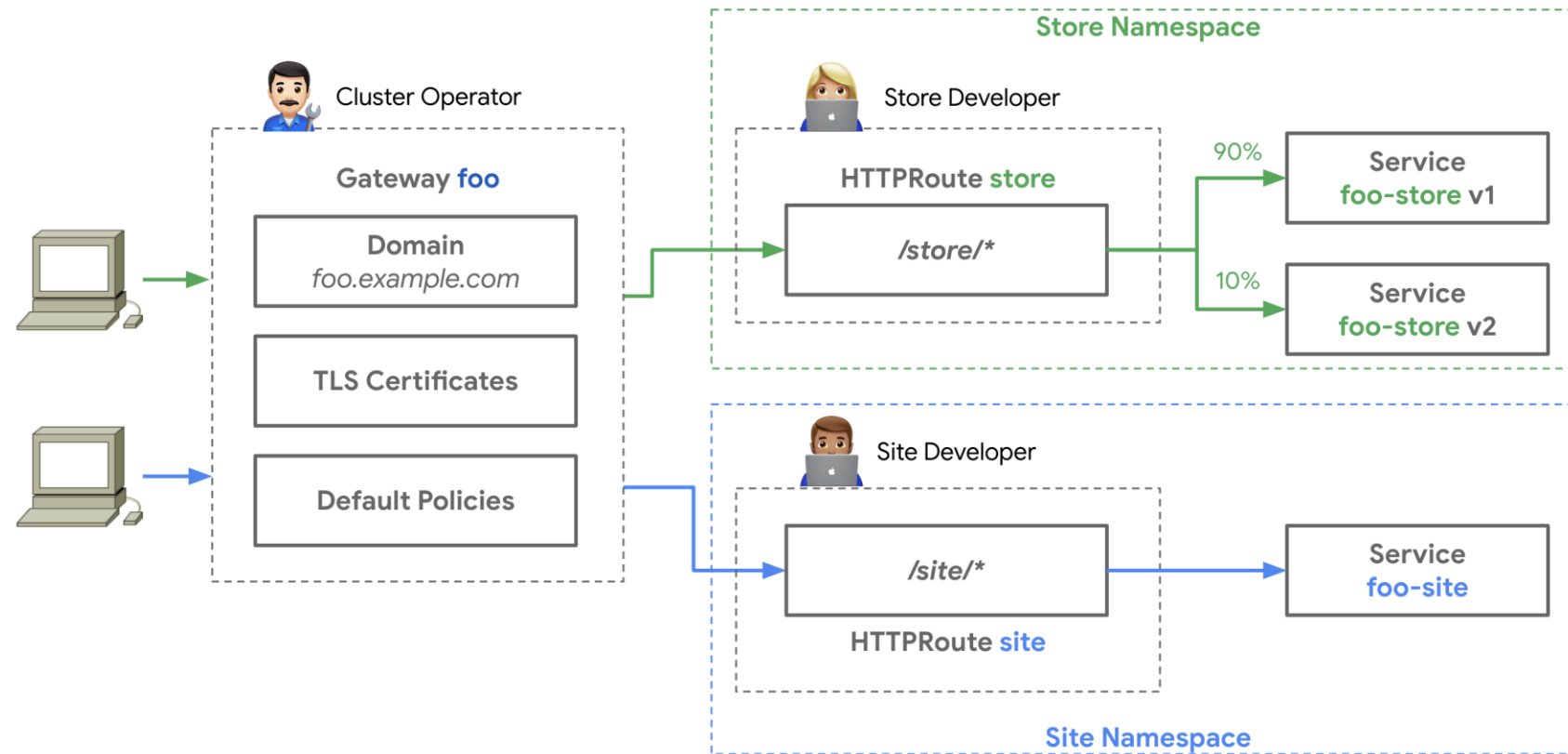
Kubernetes Gateway API

- The `Gateway` represents the instantiation of a logical load balancer and the `GatewayClass` defines the load balancer template when users create a `Gateway`.
- Route resources define protocol-specific rules for mapping requests from a `Gateway` to Kubernetes Services. `HTTPRoute` is for multiplexing HTTP or terminated HTTPS connections.



Note: Gateway API has been designed to enable granular authorization for each role in a typical organization (*segregation of duties*).

Kubernetes Gateway API

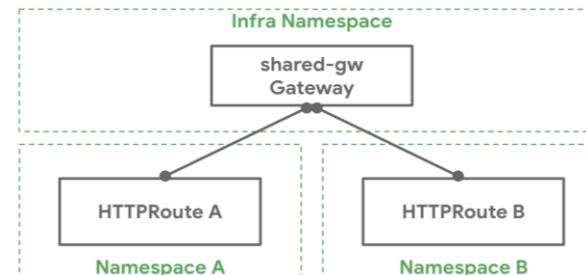




Kubernetes Gateway API: Example

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: my-gateway-class
spec:
  controllerName: gatewayclass-controller
```

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: ingress
  namespace: ingress-gateway
spec:
  gatewayClassName: my-gateway-class
  listeners:
    - name: http
      hostname: "*.86-119-44-211.nip.io"
      port: 80
      protocol: HTTP
      allowedRoutes:
        namespaces:
          from: All
```



```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-app
spec:
  parentRefs:
    - name: ingress
      namespace: ingress-gateway
  hostnames:
    - my-app.86-119-44-211.nip.io
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: my-app
          port: 3000
```

Container Storage Interface

Physical Storage

- Storage defined outside the cluster

PersistentVolume (PV)

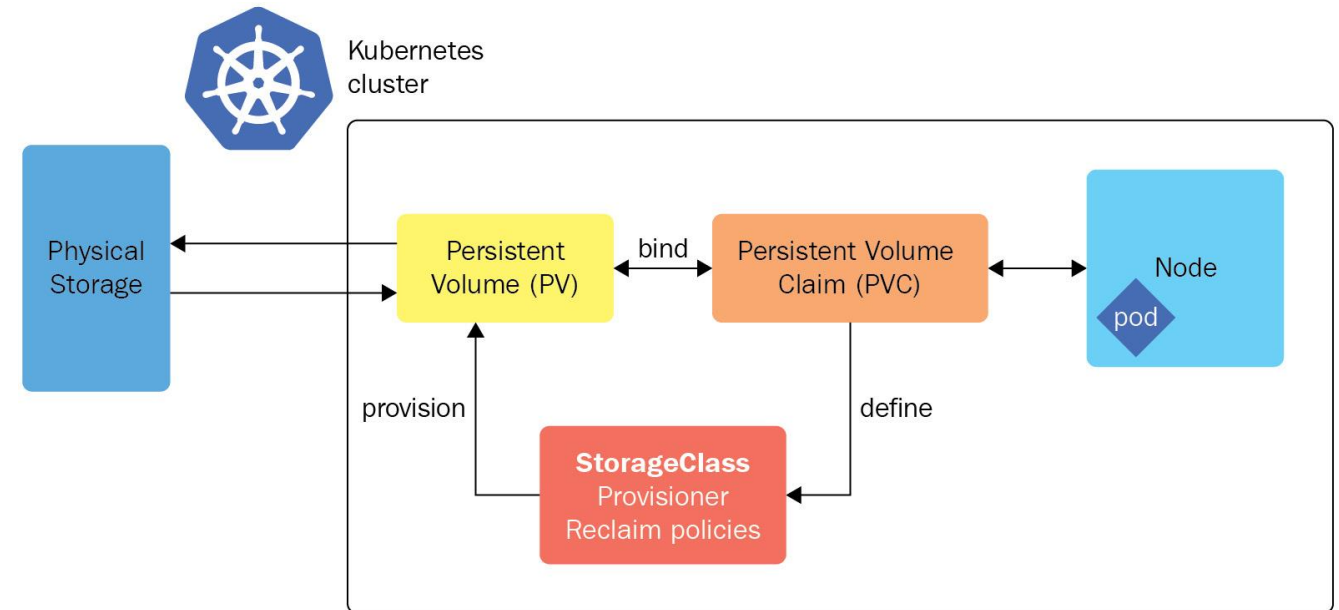
Cluster-wide specific volume

PersistentVolumeClaim (PVC)

- Namespace-specific claim for persistent volume

StorageClass

- Concrete implementation of possible kind of persistent volume



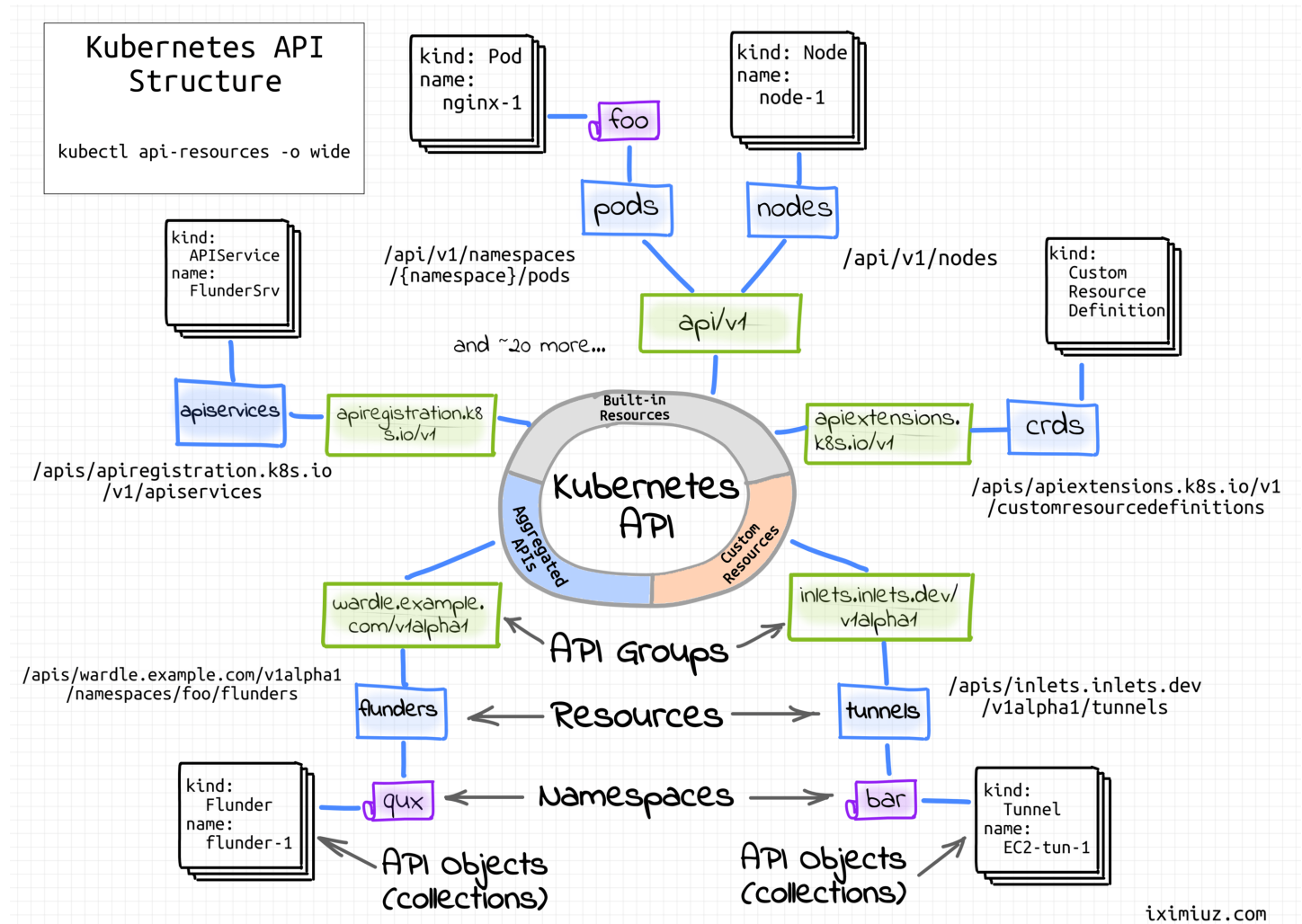
Custom Resources

API Structure bases on Resource Definitions to be provisioned / set via declarative approach.

This applies for all resources (Pods, Deployments, StorageClass, Custom Resources, etc.)

Benefits:

- To be provisioned via CLI, Yaml & REST
- Adaptable, extensible, flexible
- Lifecycle for resources via versioning



Platform 3: K8s Tutorial

- Part 1: Installation of Kubernetes via K3S.
- Part 2: Container Sandboxes (gVisor)
- Part 3: Installation of Vanilla Kubernetes with one main node and one worker node

https://dps.pages.fhnw.ch/module/cloud/platforms_to_build/cloud-reports/03-orchestrator/readme.html



Helm

Helm helps manage Kubernetes applications by enabling the definition, installation, and upgrading of even the most complex applications, using Helm charts that are easy to create, version, share, and publish.

A **Helm chart** is a package that contains all the necessary files, including templates and configuration values

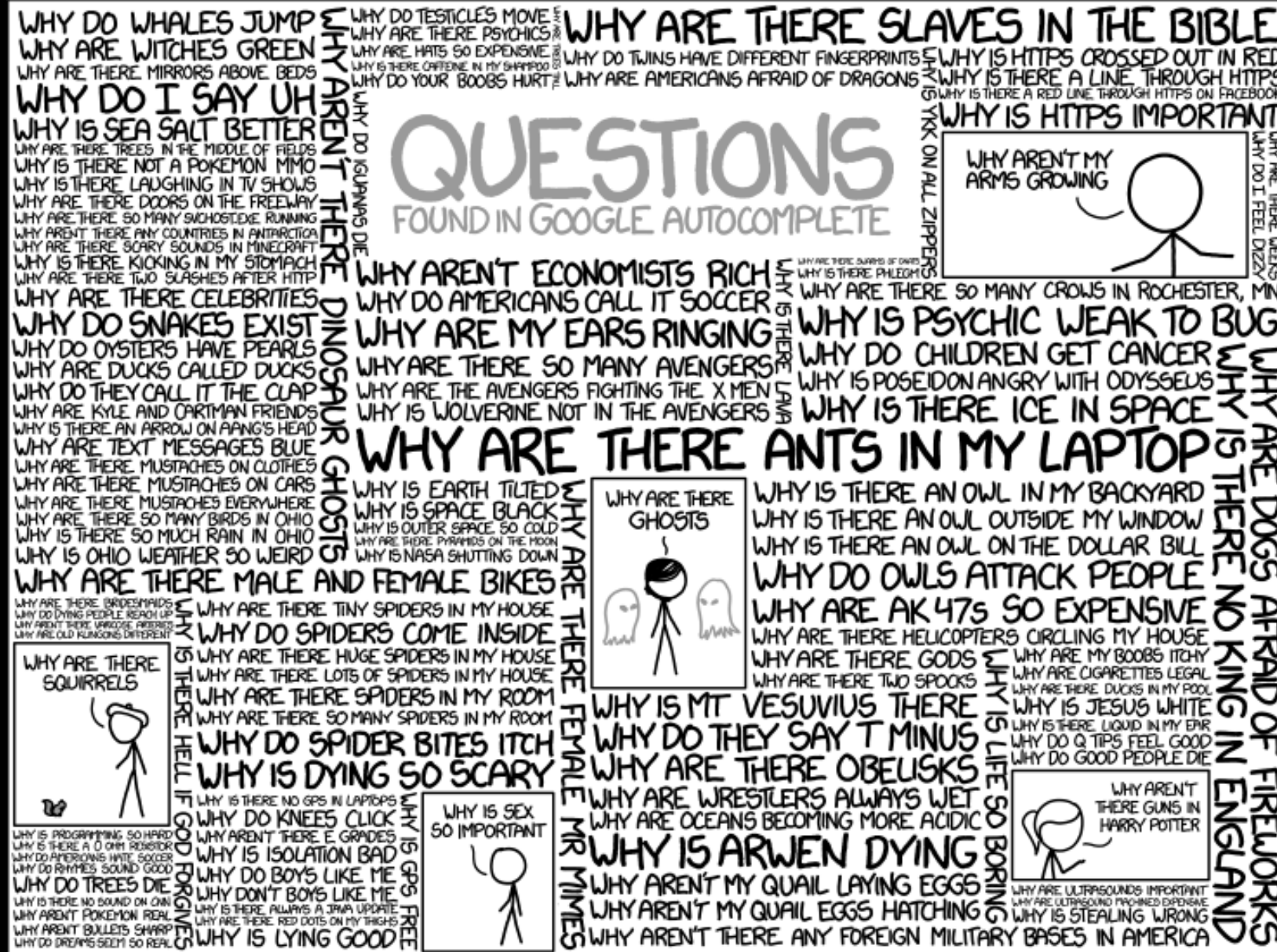
configmap.yaml

```
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
  drink: {{ .Values.favoriteDrink }}
```

Templates define Kubernetes YAML manifests as a Go template, which allows for the customization of application deployments by incorporating values from the **values.yaml** file and enabling conditional rendering.

values.yaml

```
favoriteDrink: coffee
```



Summarizing Questions

- Why does Kubernetes offer different kinds of network-implementations?
- What is the outcome if not CNI-Interface is provided?
- Name the different kinds of Storage applicable to Pods and their purpose.
- Why do different CNI-implementations exists?
- Choose one CNI-implementation and argue why you are going to use it.